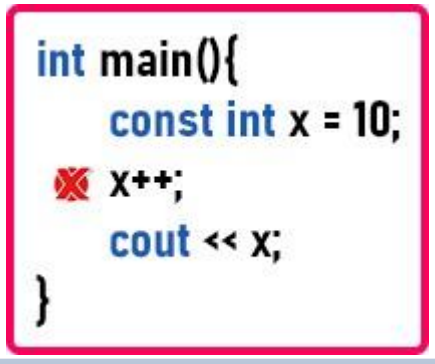
Constants in C++

As the name suggests, Constants in C++ are given to such variables or values which cannot be modified or changed once they are defined. That means they are fixed values in a program, unlike the variables whose value can be altered.

There are two ways to define constants in C++.

- By using the const keyword
- By using #define preprocessor




```cpp
#include<iostream>
using namespace std;
int main(){
    const int x = 10;
    x++;
    cout << x;
    return 0;
}
```

Output:-

These variables are also called read-only variables.

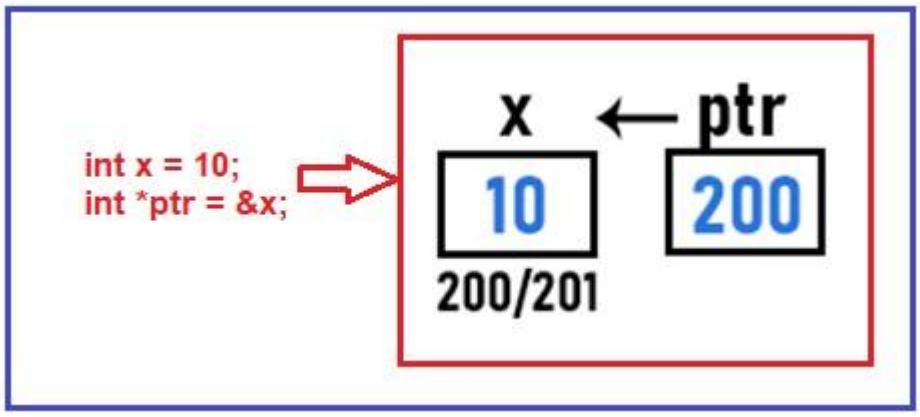
#define x 10

What is the difference between #define and const in C++?

- #define is a preprocessor directive and it is performed before the compilation process starts. But const is an identifier that will consume memory that cannot be modified.
- **#define is just a symbolic constant and const is a constant identifier.**
- **#define will not consume memory, but const will consume memory according to the data type.**
- #define is not a part of the language, it is a pre-compiler whereas const is a part of the language that is part of the compiler.

Constant Pointers in C++:-

```
int main()
{
        int x = 10;
        int *ptr = &x;
        ++*ptr;
        cout << x;
}
```



Then we have written ++*ptr. So, *ptr will increment the value of x from 10 to 11 as follows.

But

```
int main(){
        int x = 10;
        const int *ptr = &x;
        ++*ptr;
        cout << x;
}
```

In this case, the pointer ptr can point to x and **access or read x, but it cannot modify the value x.** **So here, ++*ptr is not allowed.** So, by using the constant pointer, we cannot modify the data.

This can be written in other ways also, like,

int const *ptr = &x; OR const int *ptr = *x;

Suppose we have one more variable y and it is having a value of 30 (int y = 30;). So, can we point ptr at y? **Yes. By writing, ptr = &y,** ptr will point at y. So, we can make the pointer point to some other data, but you cannot modify that. Here also, we cannot modify the value of y.

```cpp
#include<iostream>
using namespace std;
int main(){
    int x = 10;
    int y = 30;
    const int *ptr = &x;
    ptr = &y; //Allowed
    //++*ptr; //Not Allowed
    cout << *ptr;
}
//output
30
```

Another Usage of Constants in C++:

- **int * const ptr = &x;**

Now ptr is a constant pointer to an integer. So, who is the constant here? Pointer ptr is constant here. What does it mean? It means once the pointer ptr is pointing at x, it cannot be changed. After that, you will be unable to change it to another variable. **We cannot write ptr = &y** because ptr is constant. So, the address in that pointer cannot be changed.

```cpp
#include<iostream>
using namespace std;
int main(){
    int x = 10;
    int y = 30;
    int * const ptr = &x;
    ++*ptr; // Allowed
    ptr = &y; //Not Allowed
    cout<<*ptr;
}
```

Output :-

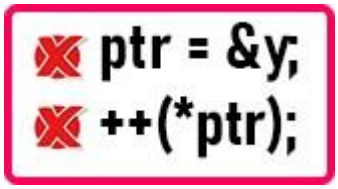
```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g
demp.cpp: In function 'int main()':
demp.cpp:8:12: error: assignment of read-only variable 'ptr'
     ptr = &y; //Not Allowed
           ^
PS F:\Desktop\CODEFORCES>
```

Two constants in C++:-

- **const int * const ptr = &x;**

Two constants in C++:-



We cannot modify the data by writing ++ *ptr and we cannot assign the pointer to another date like ptr = &y.

```cpp
#include<iostream>
using namespace std;
int main(){
    int x = 10;
    int y = 30;
    const int * const ptr = &x;
    ++*ptr; //const int * const ptr = &x; Allowed
    ptr = &y; //Not Allowed
    cout<<*ptr;
    return 0;
}
```

Output :-

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp
demp.cpp: In function 'int main()':
demp.cpp:7:8: error: increment of read-only location '*(const int*)ptr'
     ++*ptr; //const int * const ptr = &x; Allowed
       ^~~
demp.cpp:8:12: error: assignment of read-only variable 'ptr'
     ptr = &y; //Not Allowed
           ^
PS F:\Desktop\CODEFORCES>
```

There are 3 types of constant pointers in C++.

- **The pointer can be constant:** Pointer cannot be modified i.e.

  ptr = &y is not valid here.
- **Data can be constant:** Pointer cannot modify the data i.e. ++*ptr is not valid.

- **Both the pointer and data can be constant:** Here pointer cannot be modified i.e. i.e. ptr = &y is not valid, and data is also can't be modify i.e. ++*ptr is not valid.

Const Keyword in C++ Functions:-

```cpp
#include<iostream>
using namespace std;
class Demo{
public:
    int x = 10;
    int y = 20;
    void Display() const{
        x++; //Not Allowed
        cout << x << " " << y << endl;
    }
};
int main(){
    Demo d;
    d.Display();
}
```

Output:-

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp -o
demp.cpp: In member function 'void Demo::Display() const':
demp.cpp:8:10: error: increment of member 'Demo::x' in read-only object
        x++; //Not Allowed
         ^~
PS F:\Desktop\CODEFORCES>
```

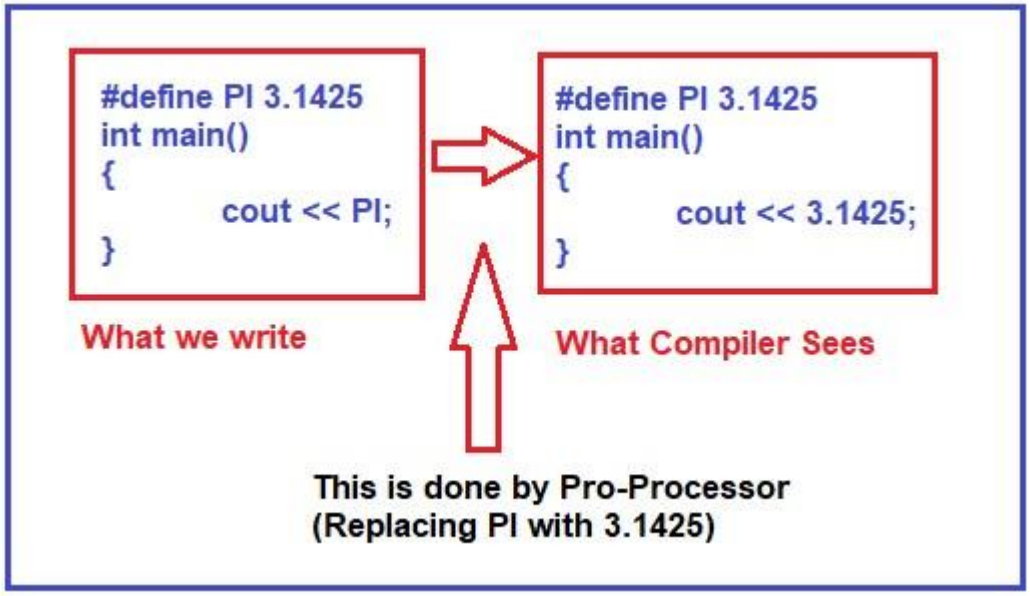
Another use of const keyword:-

```
void fun(const int &x, const int &y){
        ---
}
```

So, parameters can also be made constant. Now a and b cannot be changed through function fun. We cannot write x++ or y++ here.

Preprocessor Directives in C++

Macros or Preprocessor Directives in C++ are instructions to the compiler. **We can give some instructions to the compiler so that before the compiler starts compiling a program,** it can follow those instructions and perform those instructions. The most well-known Macros or Preprocessor Directives that we used in our program is **#define.**

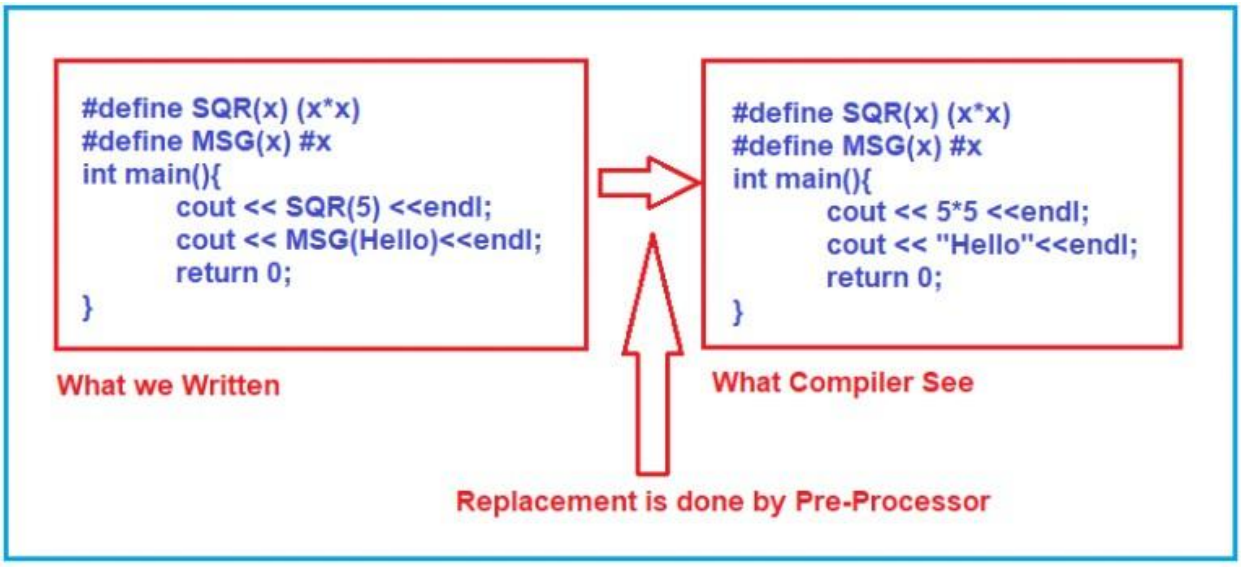


#define c cout

Now, can we write c << 10? Yes. What will happen? This c will be replaced by cout before the compilation.

Define Function using #define Preprocessor Directive in C++:-

```
#define SQR(x) (x*x)
#define MSG(x) #x
int main(){
        cout << SQR(5) <<endl;
        cout << MSG(Hello)<<endl;
        return 0;
}
```



With #x, we created another function, MSG(x). Whatever the parameters we send in MSG, that will be converted into a string.

So, if we write cout << MSG(Hello), then it means "Hello". So, we have given Hello without double-quotes. But MSG (Hello) will be replaced by "Hello".

#ifndef Directive in C++, #ifndef. It means if not defined.

```
#ifndef
        #define PI 3.1425
#endif
```

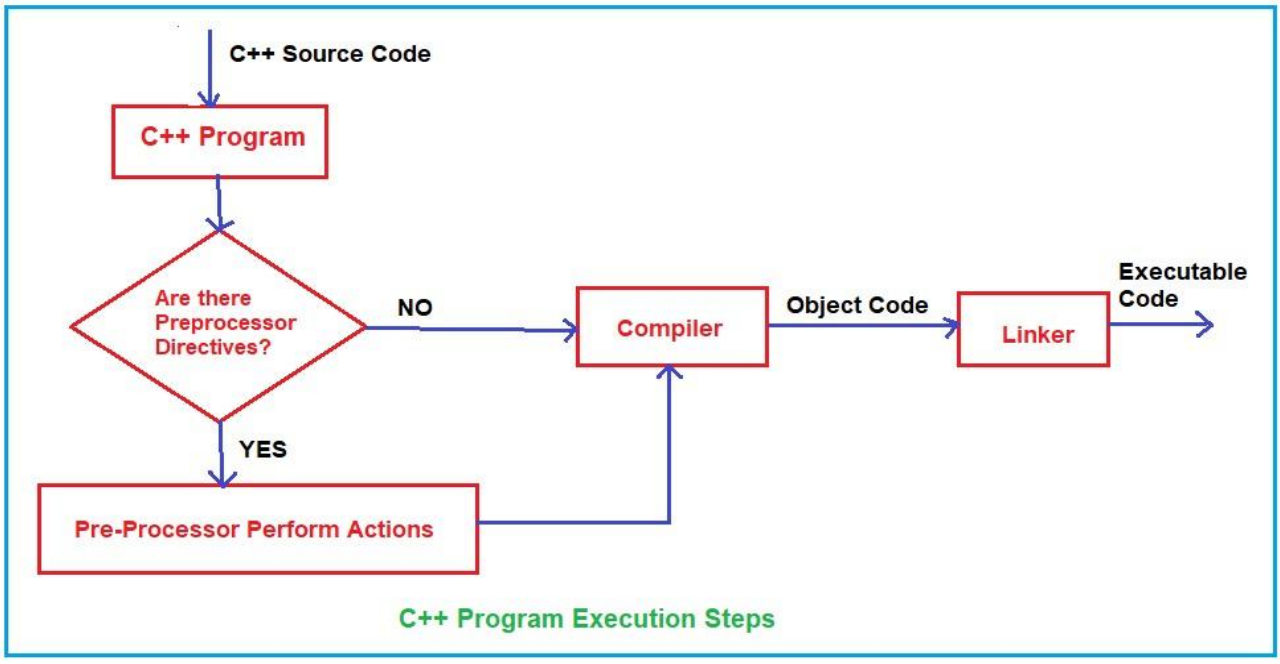
This PI is defined if it is not already defined, then only it will be defined, otherwise, it will not be defined again.

```cpp
#include <iostream>
using namespace std;
#define max(x, y) (x > y ? x : y)
#ifndef PI
    #define PI 3.1425
#endif
int main(){
    cout<<PI<<endl;
    cout<<max(121, 125)<<endl;
    return 0;
}
```

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ d
3.1425
125
PS F:\Desktop\CODEFORCES>
```

How a C++ Program Executes?



Namespaces in C++ :- Namespaces are used for removing name conflicts in C++. If you are writing multiple functions with the same name but are not overloaded, they are independent functions. They are not a part of the class.

```
void fun (){
        cout << "First";
}
void fun(){
        cout << "Second";
}
int main (){
        fun();
}
```

So, we have two functions of name fun and one main function. The main function is to call the function fun. So, which fun function will

be called? First fun or second fun? **First of all, the compiler will not compile our program. The compiler will say that we are redefining the same function multiple times.**

But we want the same function but we want to remove this ambiguity. We have to remove the name conflicts. So, for this, we can use namespaces in C++. Let us define our namespace as follows:

```
namespace First{
        void fun (){
                cout << "First";
        }
}

namespace Second{
        void fun(){
                cout << "Second";
        }
}
```

```
First::fun();
Second::fun();
```

So first we have to write the namespace, then the scope resolution operator, and then the function name.

```cpp
#include <iostream>
using namespace std;
namespace First{
    void fun(){
        cout<<"First"<<endl;
    }
}
namespace Second{
    void fun(){
        cout<<"Second"<<endl;
    }
}
int main(){
    First::fun ();
    Second::fun ();
    return 0;
}
```

Output :-

A namespace is a container that holds a set of identifiers. **These identifiers can be variables, functions, or classes,** and they are used to organize and structure code. A namespace helps to prevent naming conflicts by providing a way to group related identifiers together and give them a unique name.

using namespace First;

Now when we call the function fun anywhere in the program, it will call fun inside the first namespace. If you still want to call the second namespace function, then you can write,

Second::fun();

```cpp
#include <iostream>
using namespace std;
namespace First{
    void fun(){
        cout<<"First"<<endl;
    }
}
namespace Second{
    void fun(){
        cout<<"Second"<<endl;
    }
}
using namespace First;
int main(){
    fun();
    Second::fun();
    return 0;
}
```

Output:-